

Supplementary Material for Diagnosing and Repairing Out-of-Distribution Failure in MOBA Draft Policies

Anonymous Authors

TABLE I
MCTS TRAINING THROUGHPUT ACROSS IMPLEMENTATION STAGES.

Implementation	ep/s	Bottleneck
Python + PyTorch CPU	1.2	Forward pass dispatch
C++ tree, batched virtual loss (K=32)	5.1	Amortized sync
Full CUDA kernel (1 block=1 ep)	26	Python buffer overhead
+ ring buffer + pipelining (bring-up)	50	Warm-up, small batches
Final (steady state, campaign)	140	GPU compute

I. FUSED CUDA KERNELS FOR MCTS TRAINING

The MCTS training campaign in the main paper, 195 complete training runs spanning simulation budgets up to 800 per move and episode counts up to 1M, was only feasible because of the fused CUDA implementation described here. AlphaZero-style MCTS training is computationally demanding: each episode requires ~ 8 MCTS searches of 200 simulations each, with a neural network forward pass at every leaf node. Standard implementations incur Python interpreter overhead, CUDA kernel launch latency, and host-device synchronization for each of the $\sim 3,200$ forward passes per episode.

We developed a fused CUDA implementation that runs entire MCTS episodes inside a single kernel launch. One thread block (256 threads) executes one complete draft episode: thread 0 drives all sequential logic (UCB selection, action sampling, backpropagation) while all 256 threads cooperate on matrix multiplications during neural network forward passes via shared memory. The forward pass functions (`linear_layer`, `batchnorm_relu`, `residual_block`) are compiled as `__device__` functions callable from within the kernel with zero launch overhead. Two additional optimizations compound with the fused kernel:

- **Split backbone from heads:** Value symmetrization runs only the small value head ($257 \rightarrow 128 \rightarrow 64 \rightarrow 1$) twice rather than the full backbone, halving symmetrization cost.
- **Pipelined generation:** Episode results are written into pre-allocated ring buffers via C++ `memcpy`; a generation thread launches GPU batches asynchronously while the main thread performs gradient updates.

Table I shows the progression; all stages measured at 200 simulations per move. The first four rows are development-

time measurements taken as each optimization landed; the final row is the sustained steady-state rate of the finished implementation, measured across the full training campaign (195 complete runs, 128 concurrent episodes per kernel launch) under four-GPU load on NVIDIA RTX PRO 6000 Blackwell Workstation Edition GPUs (96 GB each). The highest rate recorded in the training logs is 161 episodes/second. Two corrections to earlier versions of this table: an earlier draft printed 10.9 ep/s for the virtual-loss stage, but 10.9 was the development log’s speedup *factor* over the sequential Python pipeline (2,145 vs. 197 ms/episode); the rate is 5.1 ep/s, and since the C++ engine was virtual-loss batched from its first commit, the two development rows collapse to one. The rows are also heterogeneous in leaf evaluation (development history, not a controlled comparison); the controlled comparison appears below. The gap between the bring-up and final figures reflects cumulative-average warm-up in early measurements and subsequent batching maturity. Sustained throughput is $117\times$ the Python baseline and scales inversely with simulation budget (74 episodes/second at 400 simulations per move), confirming the implementation is compute-bound rather than overhead-bound: a 300K-episode run completes in under 40 minutes on one GPU. Each episode requires ~ 1 MB of global memory for the MCTS tree (2,048 nodes); with 128 concurrent episodes per kernel launch, total GPU memory usage is ~ 140 MB plus 10 MB of network weights.

a) *Apples-to-apples baseline.*: For a controlled speedup claim we built a tuned host-tree baseline in the style of KataGo’s batched-evaluation pipelines: 64–256 CPU threads each running one episode’s tree, leaf evaluations batched across threads onto the GPU via cuBLAS GEMM over multiple streams, reproducing the production kernel’s exact per-simulation work (policy priors at expansion, Generic Draft rollout to terminal at every leaf, symmetrized enriched-WP scoring). At equal work *and* equal search semantics (leaf batch 1; first-turn root-visit total-variation distance from the kernel 0.047, at the 0.044 kernel-vs-kernel seed-noise floor; win rate vs. GD 0.761 vs. the kernel’s 0.757), the host pipeline sustains 68 ep/s against the fused kernel’s 151 ep/s on the same GPU: the fused kernel is $2.2\times$ faster, the figure quoted in the main paper, because per-simulation host-device round trips cannot be amortized without within-episode leaf batching. Virtual-loss leaf batching does push the host pipeline to 163 ep/s ($1.07\times$ the kernel), but only by degrading the search itself: at leaf

TABLE II

STRENGTH-ADJUSTED VALIDITY OF THE INTERACTION METRICS: ACTUAL WIN RATE IN THE TOP MINUS THE BOTTOM METRIC DECILE (PP), AND THE NUMBER OF INCREASING ADJACENT DECILE STEPS (OF 9). DECILES ARE COMPUTED WITHIN STRATA OF SKILL TIER $\times$ OWN-TEAM $\times$ OPPOSING-TEAM MEAN-HERO-WR QUINTILES AND COUNT-WEIGHTED ACROSS STRATA (95% CIs ABOUT ± 1.1 PP ON THE TEST SET AND ± 0.4 PP POST-SNAPSHOT). “SELF-INCL.”: THE STATISTICS CONTAIN THE SCORED GAMES’ OWN OUTCOMES.

Statistics	Games	Synergy	Counter	Self-incl.
External	test (77,962)	+10.6, 9/9	+9.7, 8/9	yes
External	post (583,674)	+9.1, 9/9	+8.0, 9/9	no
Own train split	test	+9.0, 8/9	+13.5, 8/9	no
Own train split	post	+7.8, 9/9	+12.7, 9/9	no

batch 32 the win rate vs. GD falls from 0.757 to 0.531 and the visit-distribution TV rises to 0.62; softening virtual loss to 0.25 recovers only 0.712.¹ Per-component profiling shows the GD rollout evaluation dominates ($\sim 75\%$ of the host pipeline’s evaluator wait time; policy priors and WP scoring share the rest), and that the kernel’s per-node cache of opponent GD distributions is dead code in practice: leaves expand only at our-team steps, so cached nodes are never re-traversed at opponent steps (measured cache-miss count 0), which is why it is not listed among the optimizations above. The remaining advantages of the fused kernel are operational: 0.12 s startup, a deterministic ~ 270 MB footprint, and no host thread pool or Python runtime at inference.

II. SYNTHETIC AUGMENTATION SWEEP: UNSEEN VS. SPARSE SCOPE

We swept 27 configurations, re-run with leak-free models (256 $\rightarrow$ 128, out-of-fold features, own composition table, seed 42): assigned win rate for unseen compositions (10/20/30%), volume (50/100/200 records per composition and tier), and scope (unseen only; sparse only, which tops up observed compositions with fewer than 300 games and win rate below 48% to 300 games at their own win rate; or both). Sparse-only augmentation does not depend on win rate or volume (19,107 records in every cell) and leaves the five-tank WP at 0.26, against 0.27 without augmentation. Unseen-only augmentation lowers it to 0.07–0.12 at 10%, 0.16–0.18 at 20%, and 0.23–0.28 at 30%; adding the sparse records changes little (0.08–0.10 at 10%). Held-out accuracy stays at 57.5–57.7% and post-snapshot calibration slope at 0.93–1.02 in every cell, and 24–27 of 28 sanity tests pass. The active ingredient is the unseen compositions, and lower assigned win rates penalize five tanks more.

III. VALIDITY OF THE INTERACTION METRICS

The main paper’s interaction-awareness metrics are built from the pairwise synergy and counter deltas (Section III of the main paper), which subtract from each observed pair win

rate an independence expectation formed from the two heroes’ individual win rates. We validate that these deltas carry real outcome signal by scoring each side of real drafts with the exact metric implementations against the game’s actual outcome. Pooled naively, the decile curves are not monotone: subtracting the individual-strength expectation makes the deltas strongly anti-correlated with team hero strength ($r = -0.87$ between team synergy delta and team mean hero win rate), so teams of individually strong heroes receive negative deltas while still winning more. We therefore control for that component, computing deciles within tier and own/opposing team-strength strata.

The submission ran this check on the 38,981 test replays with the external statistics, which contain those replays’ own outcomes (first row of Table II). On 291,837 post-snapshot games, which were uploaded after the statistics were pulled and so cannot be in them, the same statistics give spreads of +9.1pp (synergy) and +8.0pp (counter), both monotone across all nine steps. Self-inclusion inflated the published spreads by about 14% and 18%. Statistics built only from our training split give the same qualitative result on both game sets. The interaction deltas measure an outcome-relevant property of a composition beyond individual hero strength, which is the component they are designed to isolate.

IV. DUAL-SNAPSHOT STABILITY OF THE INTERACTION METRICS

The main paper notes (Section III) that the interaction-metric magnitudes are snapshot-sensitive: re-pulling the community aggregate statistics shifts the pairwise deltas systematically, so absolute synergy and counter values are not comparable across snapshots. The collapse diagnosis and the method comparisons rest instead on *relative* gaps. Here we verify directly that the relative ordering of strategies is stable across statistics snapshots.

The twelve MCTS training configurations of the main paper’s simulation-scaling study were each re-benchmarked twice under an identical protocol (15 seeds $\times$ 600 drafts against behavioral-cloning opponents): once with the aggregate statistics in use before the May 2026 re-pull, and once with the frozen 2026-05-19 snapshot used throughout the paper. The trained policy networks are identical in both arms; only the statistics feeding the enriched features and the metric lookups change. Table III shows synergy exploitation and counter responsiveness per configuration under both snapshots.

The ordering is essentially unchanged: Spearman rank correlations across the twelve configurations are 1.00 for synergy, 0.97 for counter responsiveness (adjacent reshuffling among near-tied configurations), and 1.00 for early-pick resilience. Absolute values shift by at most +0.05 (synergy) and ± 0.007 (counter), small relative to the between-configuration gaps the paper’s conclusions rest on (e.g., the +0.9 synergy separation between deep-search and anchored strategies). This benchmark predates the paper’s final retrained models, so its absolute magnitudes differ from the main-paper tables; the comparison isolates the snapshot effect, which is the quantity of interest. Strategy-level rich evaluations (Table V of the main paper)

¹The equivalence check also uncovered a virtual-loss bookkeeping bug in the historical batched implementation of Table I: the virtual-loss undo was applied to the root node as well as the descended path, driving the root visit count negative, at which point $\sqrt{\text{negative}} = \text{NaN}$ poisoned UCB selection from the second leaf batch onward.

TABLE III

INTERACTION METRICS FOR TWELVE MCTS CONFIGURATIONS UNDER TWO STATISTICS SNAPSHOTS (EARLIER PULL VS. FROZEN 2026-05-19; 15 SEEDS $\times$ 600 DRAFTS EACH, IDENTICAL POLICIES). ROWS SORTED BY FROZEN-SNAPSHOT SYNERGY. SPEARMAN RANK CORRELATION OF THE CONFIGURATION ORDERING ACROSS SNAPSHOTS: $\rho = 1.00$ (SYNERGY), $\rho = 0.97$ (COUNTER), $\rho = 1.00$ (EARLY RESILIENCE, NOT SHOWN).

Config	Synergy		Counter	
	Earlier	Frozen	Earlier	Frozen
L_800sim_4M	+1.20	+1.21	+0.19	+0.18
J_800sim	+1.07	+1.07	+0.20	+0.20
I_600sim	+0.99	+1.01	+0.20	+0.19
F_400sim	+0.92	+0.94	+0.18	+0.19
E_1M	+0.91	+0.94	+0.18	+0.18
H_augmented	+0.73	+0.75	+0.16	+0.16
B_fullwp	+0.69	+0.73	+0.15	+0.15
A_partial	+0.68	+0.72	+0.14	+0.14
K_truebase	+0.54	+0.59	+0.13	+0.12
G_base	+0.53	+0.57	+0.11	+0.11
C_large	+0.45	+0.49	+0.13	+0.13
D_deep	+0.14	+0.14	+0.12	+0.11

were run only under the frozen snapshot; the earlier pull existed as a live database state and was not retained, so the controlled dual-snapshot evidence above is the MCTS configuration family, which spans the full range of interaction-awareness the paper reports.

V. EXTERNAL VALIDATION ON COORDINATED-LEAGUE DRAFTS

The main paper’s behavior policy is uncoordinated ranked-ladder play, and its Limitations note that every evaluation shares that corpus. As an external test, we evaluate the frozen snapshot-trained win-probability models on drafts from the Nexus Gaming Series (NGS), an amateur organized league in which persistent five-player teams draft together with dedicated captains. Scanning the Heroes Profile NGS archive yielded 11,132 games, of which **11,034** pass filtering (patch 2.55, complete 16-step draft, known winner), spanning 2021–2026 across ten season clusters, the same patch era as our ladder corpus.

a) Coordinated drafts are more meta-concentrated.: A natural hypothesis is that coordinated teams, freed from solo-queue comfort picking, explore more of the hero pool. The data show the opposite, at both the hero and the composition level. Hero level: NGS pick entropy is 5.92 bits vs. the ladder’s 6.25 (effective pool 60.5 vs. 75.9 heroes), the top-10 heroes take 33.5% of picks vs. 25.1%, and the off-meta share (picks outside the ladder top-30) is 39.8% vs. 43.3%; hero pick-rate rank correlation between the populations is 0.74. Composition level: NGS fields 78 distinct role compositions vs. 238 on ladder, with composition entropy 2.69 vs. 3.06 bits, the top-5 compositions covering 84.3% vs. 78.6% of teams, and 90% coverage reached in 9 vs. 12 compositions. The modal composition is the same standard core in both populations (bruiser/healer/double-ranged/tank: 53% vs. 50%); NGS’s second-most-common composition is a double-tank variant (12.3%). Our reading: coordinated teams converge on a known-good playbook, while ladder “diversity” partly reflects uncoordinated comfort picking and error. This corroborates the

main paper’s characterization of the behavior policy as shaped by social dynamics rather than win maximization: removing the coordination constraint is not what diversifies drafting; removing strategic discipline is.

b) Win-probability transfer.: Scored at the game level with symmetrized predictions, the leak-free enriched model reaches 55.5% accuracy on NGS (95% CI ± 0.9 pp) against 57.7% on the held-out ladder test set; the leak-free naive and augmented models also score 55.5%. NGS games are in neither our corpus nor the Storm League aggregates, so no model has self-inclusion here. The ladder-to-NGS drop is about 2.2pp for every model. The submission compared NGS with a leaky ladder figure (58.1%) and reported a larger drop for the enriched model; its enriched model also scored 55.5% on NGS, with calibration slope 0.75 against 0.86 for the leak-free enriched model and 0.94 for the leak-free naive model. Splitting NGS drafts into quartiles by off-meta pick share, the submitted enriched model scored 57.6% on the most-meta quartile and 54.5% on the least-meta quartile. The leak-free models show no such trend: the enriched model scores 56.0%, 54.2%, 56.9%, and 54.8% from most- to least-meta quartile (about 2,760 games each, CI ± 1.9 pp), and the naive model 56.4%, 54.7%, 56.2%, and 54.6%. We therefore do not claim that the degradation concentrates in off-meta drafts. 99.98% of NGS team compositions fall within the composition sets observed in the ladder-era aggregate statistics, so this shift is a re-weighting within familiar structures, not entry into the never-observed region that the paper’s synthetic augmentation targets. One caveat caps interpretation: NGS outcomes also reflect persistent team-strength imbalances that the models never observe (the same organized teams meet repeatedly), which lowers attainable accuracy in a way not directly comparable to solo-queue ladder play.

VI. DRAFT CONCENTRATION AND OUT-OF-DISTRIBUTION EXTRAPOLATION

How far do the agents drift from human play? We measure the leak-free operating-point agent (F_oof, 5 seeds $\times$ 1,000 benchmark drafts, mid label, 200 inference simulations, root temperature 1) and, for comparison, the submission’s J_800sim (5 seeds).

a) Concentration: Each F_oof seed uses 24–27 distinct heroes at 3.25–3.59 bits of pick entropy; pooling the five seeds gives 3.58 bits, so the seeds converge on one head. Its seven favorite heroes are Samuro, Rehgar, Hogger, Deathwing, Falstad, Illidan, and Ragnaros; the submitted J_800sim’s head overlaps heavily (Hogger, Samuro, Rehgar, Deathwing, Auriel, Illidan, Leoric) and is slightly more diverse (29–32 heroes, 3.74 bits pooled).

b) Are the favorites’ interactions real?: For all 21 pairs and 35 trios of the seven favorites we compare the win rate of real teams containing them with the independence expectation from the heroes’ own win rates (the paper’s synergy convention). In our training split the favorite pairs beat independence by +0.54pp ($n = 228,342$ team-games, $z = 5.2$) and the trios by +1.70pp ($n = 12,767$, $z = 3.8$). On 450,445 post-snapshot games (including the drifted 2.55.17 builds), which no model

TABLE IV

PER-DRAFT VARIANCE OF THE 20-MEMBER LEAK-FREE ENSEMBLE.
RATIO: MEDIAN RELATIVE TO HELD-OUT HUMAN DRAFTS; >P95: SHARE
OF DRAFTS ABOVE THE HUMAN 95TH PERCENTILE.

Draft set	Med. $\times 10^3$	Ratio	>p95
Held-out human ($n=2000$)	0.43	1.00	(5.0%)
F_oof agent ($n=2000$)	0.52	1.21	7.1%
Synthetic degenerate ($n=500$)	1.36	3.20	54.0%
Degenerate probes ($n=6$)	2.57	6.05	5/6
<i>Tournament drafts per strategy ($n=2000$ each)</i>			
G&A estimator	0.94	2.21	35.5%
MCTS	0.77	1.82	22.9%
Constrained greedy	0.77	1.81	24.6%
MCTS, no features	0.77	1.80	24.9%
Enr.+aug greedy	0.76	1.79	24.2%
Enriched greedy	0.74	1.75	23.8%
Constrained MCTS	0.71	1.66	20.2%
MCQ	0.63	1.48	20.9%
CQL enriched	0.56	1.32	15.6%
GD	0.55	1.28	13.8%
CQL naive	0.54	1.27	15.7%
G&A discriminator	0.46	1.09	11.2%

saw, the pairs beat it by +0.21pp ($n = 56,029$, $z = 1.0$) and the trios by +1.68pp ($n = 3,169$, $z = 1.9$). The trio edge replicates in size but not yet in significance; the pair edge shrinks by more than half. For the submitted agent’s favorites the forward edges are similar (+0.26pp pairs, +1.35pp trios, both not significant). The submission’s statement that search “exploits precisely the interactions the data certifies”, based on in-sample checks only, is withdrawn.

c) *Where the agent leaves the data:* None of F_oof’s ten most frequent five-hero teams (together 38.8% of its drafts) occurs even once in the 1.95M-game corpus. One level up it stays in familiar territory: 98.7% of its drafts land on role compositions admitted to our own composition table (at least 50 games in the tier label) and 63.7% on compositions with at least 1,000 games.

VII. ENSEMBLE EPISTEMIC UNCERTAINTY OF THE VALUE MODEL

Do functionally different value models agree on the drafts the tournament judged? We train a 20-member leak-free ensemble on our out-of-fold features (validation early stopping): 8 full enriched models varying architecture (256→128 or 512→256→128), training-data half, and seed; 9 models each dropping one enriched feature group; and 3 identity-only models. For each draft we compute the variance of the 20 symmetrized win probabilities.

The operating-point agent’s benchmark drafts draw near-human disagreement ($1.21\times$ the human median, 7.1% beyond the human 95th percentile). Degenerate drafts are sharply elevated ($3.2\times$ for synthetic unseen compositions, $6.0\times$ for the fixed probes). Every strategy’s tournament drafts sit between $1.09\times$ and $2.21\times$, below the degenerate regime; the anchored strategies are priced most confidently and the strategies that search a value function off the human manifold highest. The tournament’s verdicts were therefore issued where the

TABLE V

HELD-OUT QUALITY ON HALF B OF VALUE FUNCTIONS TRAINED ON HALF A. “OWN-SPLIT VAL.”: WHAT A LEAKY PIPELINE REPORTS ABOUT ITSELF.

Proxy statistics	Games	Own-split val.	Held-out	Slope
in-sample, 1/8 of A	122K	68.4%	53.2%	0.17
in-sample, all of A	975K	59.4%	56.1%	0.61
out-of-fold, all of A	975K	56.3%	57.0%	1.10
hero identity only	975K	56.5%	57.1%	1.05

model family mostly agrees, with the least agreement on the Gourdeau estimator’s drafts (35.5% beyond the human 95th percentile).

VIII. LEAKAGE MECHANISM: A WITHIN-APSHOT SPLIT EXPERIMENT

To isolate how in-sample statistics affect a value function and the search that optimizes it, without any balance drift, we split the pinned snapshot by a salted replay-level hash into halves A and B. Proxies, their statistics, their early stopping, and any search against them use only half A; references are built only from half B (three out-of-fold enriched judges, a hero-identity judge, and a cross-fitted realized-outcome index), plus the post-snapshot index and the Quick Match judge.

Table V shows the mechanism. In-sample statistics make the value function overconfident on games outside them (slope 0.61 at 975K games, 0.17 at 122K) and less accurate, while reporting inflated accuracy on its own validation rows. Out-of-fold statistics remove the effect. Width, depth, dropout, weight decay, and training length each move held-out accuracy by less than 0.2pp; leakage dominates every capacity knob.

Search then exploits the error. With a behavioral-cloning prior and argmax root selection, raising search from 1,024 to 16,384 simulations lifts the in-sample proxy’s own score from 0.780 to 0.848 and its half-B judges from 0.631 to 0.666; the out-of-fold proxy, whose own score ($0.673 \rightarrow 0.734$) tracks the judges almost one for one, reaches 0.661 $\rightarrow$ 0.713 on the same judges and higher values on every realized index. A small in-sample proxy (1/8 of A) shows classic over-optimization: its own score climbs to 0.932 while the half-B judges peak at 4,096 simulations and then fall. A leak-free proxy trained on 1/8 of the data beats an in-sample proxy with $8\times$ the data on both realized indices. In the paper’s own self-play pipeline on half A (2 seeds per cell), raising training simulations from 200 to 800 at matched episodes moves the in-sample proxy’s references down by 0.02–0.05 and the out-of-fold proxy’s references up by 0.01–0.03.

Two diagnostics that did not detect the problem: the per-draft disagreement of a 4-seed ensemble trained on the same statistics stays between 0.0053 and 0.0067 at every search pressure (seeds share the leak), and pessimistic lower-confidence-bound search using that disagreement moves the references by at most 0.007. Held-out calibration slope did detect it, which is why the main paper reports it for every value function and judge.

TABLE VI

HELD-OUT ACCURACY AND CALIBRATION SLOPE OF EACH REFERENCE ON GAMES NONE OF THEM TRAINED ON: THE PAPER’S 38,981 TEST REPLAYS, AND 158,608 GAMES ON THE DRIFTED 2.55.17 BUILDS (R17 IS FIT ON THOSE GAMES AND OMITTED THERE).

Reference	Test (snapshot)		2.55.17	
	Acc.	Slope	Acc.	Slope
gN (3 OOF enriched judges)	55.3	0.97	55.2	0.88
gN-naive	56.0	0.91	55.8	0.83
RN (realized index)	54.7	1.04	54.7	0.96
R17 (realized index, drifted)	53.7	0.72	—	—
QM2026	53.9	0.44	53.7	0.43
QM2021	54.5	0.39	52.4	0.24
Leak-free naive (in corpus)	57.9	1.20	56.3	0.92

IX. CALIBRATION OF THE INDEPENDENT REFERENCES

The post-snapshot judges and the realized index are close to calibrated on snapshot games. The Quick Match judges are strongly overconfident on ranked drafts (slopes near 0.4): their probabilities are too extreme by a factor of about 2.5, so the main paper uses them for ordering only. All references are well below the 57.7% of an in-corpus model, the price of independence: smaller training sets, a different time window, or a different game mode.

X. FEATURE-GROUP SWEEP (FRACTIONAL FACTORIAL)

The submission’s sweep trained all 2^{10} subsets of ten feature groups at two architectures and two learning rates (4,096 configurations) with features that contained the test games’ outcomes. The revision re-runs the question with leak-free statistics using a 2^{10-3} fractional factorial: base factors `map_type`, `role_counts`, `hero_wr`, `team_avg_wr`, `hero_map_wr`, `map_delta`, `pairwise_counters` (A–G), and generated factors `pairwise_synergies` = ABCD, `counter_detail` = ABEF, `synergy_detail` = ACEG. Every word of the defining relation has length at least 5 (resolution V), so main effects are unaliased with each other and with all two-factor interactions, and each main effect equals the full-factorial marginal the submission reported, up to four-factor and higher interactions. Each of the 128 configurations trains one 256→128 model (seed 42, learning rate 5×10^{-4} , validation early stopping). Effect SE, from the residual after fitting main effects: 0.02pp (test accuracy), 0.01pp (post-snapshot accuracy), 5×10^{-5} (post-snapshot log-loss).

The four pairwise groups lose 0.12–0.25pp each relative to the submission, which is where self-inclusion was concentrated (the submission also averaged over two architectures and two learning rates, a smaller difference). Role counts, which read no statistics, keep their effect. Every effect is below 0.15pp in absolute value on both held-out sets.

XI. OFFLINE RL: SWEEPS AND ROBUSTNESS

The main paper’s Table V shows one or two cells per offline-RL family. The full sweeps, run on the pinned snapshot with naive (identity) features unless noted, all end at an anchored policy. CQL across $\alpha \in \{0.1, 0.5, 1, 2, 5\}$ (5 seeds $\times$ 1000

TABLE VII

MAIN EFFECT OF EACH FEATURE GROUP (WITH MINUS WITHOUT), LEAK-FREE STATISTICS. SUBMISSION: THE REPORTED FULL-FACTORIAL MARGINAL WITH LEAKY STATISTICS.

Group	Test	Post	Post LL $\times 10^3$	Submitted
<code>role_counts</code>	+0.10	+0.14	−0.68	+0.16
<code>pairwise_counters</code>	+0.08	+0.05	+0.06	+0.20
<code>map_delta</code>	+0.01	+0.04	+0.01	+0.09
<code>pairwise_synergies</code>	−0.02	+0.01	+0.08	+0.15
<code>map_type</code>	+0.00	−0.01	−0.01	+0.01
<code>team_avg_wr</code>	−0.03	−0.03	+0.13	−0.05
<code>synergy_detail</code>	−0.09	−0.04	+0.26	+0.16
<code>counter_detail</code>	−0.06	−0.06	+0.61	+0.14
<code>hero_map_wr</code>	−0.10	−0.09	+0.48	−0.02
<code>hero_wr</code>	−0.11	−0.13	+0.42	−0.16

drafts each): 93.0–99.9% healer and 2.2–13.4% degenerate, best at $\alpha = 2$ (99.8%, 2.2%). BC-regularized CQL across $\beta \in \{0.1, 0.5, 1, 2\}$: near-perfect safety, high GD agreement, negative synergy. MCQ across $\tau \in \{0.4, \dots, 0.8\}$: for $\tau \geq 0.6$ the penalty is inactive and the policy uses the same ~ 9 heroes regardless of context; $\tau = 0.5$ gives the dead-network policy of the main text (all 256 and 128 units of hidden layers 2 and 3 output zero on 200,000 held-out states; at $\tau = 0.6$ layer 3 is fully dead; a CQL $\alpha = 1$ control has 22 and 16 dead units). IQL across a $\tau \times \beta$ grid of six cells: statistically identical cells, advantage weights 0.998–0.999, Q-network held-out loss 0.6948 against $\ln 2 = 0.6931$ for a constant.

Capacity and target-update robustness. At $\alpha = 1$ we crossed three network sizes (256→128→64, 512→256→128, 1024→512→256) with five target-network update rates ($\tau_{\text{target}} \in \{0.001, 0.005, 0.01, 0.05, 0.1\}$; 200 drafts each). Across all 15 configurations the collapse signals are stable: healer 98.0–100%, degenerate 0.5–2.5%, 43–52 distinct heroes, entropy 4.25–4.52 bits, GD agreement 69.0–77.3%, synergy −0.08 to +0.02. No configuration is interaction-aware, and the update rate is inert, as expected with Monte Carlo returns (no bootstrapping).

CQL with enriched features. Adding the 86 enriched features (375-dim state) makes CQL slightly safer (2.3–2.5% degenerate) and changes nothing else: synergy −0.16 to −0.20, entropy 4.3–4.5 bits, GD agreement 68.6–73.7% vs. 70.4–72.3% for naive CQL. These models were trained with the submission’s external statistics and were not retrained leak-free; their collapse comes from the anchoring penalty, which does not depend on feature accuracy.